

My React Project - JavaScript vs TypeScript

Week 3 - 25/06/2026 Thursday

The Difference Between TypeScript And JavaScript

TypeScript is derived from JavaScript, and if we take a JavaScript file and simply rename it into .ts, it will likely be able to run without any problems. However, there are key differences between them, which I will illustrate in a table below:

Key Difference	JavaScript	TypeScript
Type System	Dynamic	Static
Execution	Runs directly	Must be compiled into JavaScript first
Error Handling & Detection	Errors are found after running the program (run time)	Errors are found before running the program (compile time)
Where It's Used	Small, simple projects	Large, complicated projects

What is the file type for each JS and TS?

1. The file type for JavaScript is .js
2. The file type for TypeScript is .ts

Week 4 - 29/06/2026 Monday

Phase 1

Introduction

Root em and pixels are commonly used units in web design to measure the font size. Although it can seem like both measuring units provide the same output, they're actually very different in terms of web accessibility, and each has its own key points, and preferred approaches.

Pixels are considered absolute units, meaning that 1px == 1px. This inflexibility limits the web accessibility in terms of responsive design, meaning that it doesn't adapt to the user's preferences.

On the other hand, root em is adept and is very flexible when it comes to responsiveness in web accessibility and even small actions such as zooming in.

1rem == 16px.

Part 1. Components, Props, & States:

Components, props and states are the foundational building blocks of any React project:

1. Components: reusable and independent UI building blocks.
2. Props: fixed (immutable) data passed from parent to child.
3. State: unfixed (mutable) data managed within a component that triggers UI renders.

Functional components are the basic building blocks in React used to create and manage UI elements.

Props to pass data between components properties similar to function arguments in JavaScript and attributes in html. (optional props are those not required to return).

Example:



```
main.jsx | index.html |
import { createRoot } from 'react-dom/client'

function Car(props) {
  return (
    <h2>I am a {props.brand}</h2>
  );
}

createRoot(document.getElementById('root')).render(
  <Car brand="Ford" />
);
```

(https://www.w3schools.com/react/showreact.asp?filename=demo_react_props)

useState hook is a built-in React function that lets you add and manage local state within functional components. It tracks data changes across component re-renders.

When a user updates a useState variable, React automatically updates the UI accordingly.

Event handling and conditional rendering are the two foundational pillars that turn static React components into dynamic, interactive user interfaces.

The key difference:

1. Event handling captures the user's intent.
2. Conditional Rendering updates the visual display based on that intent.

The key concept is **Component Reusability**, which is a software development practice where distinct, self-contained (independent) pieces of code and UI elements are built to be used across multiple applications or various parts of the same project.

This practice has many benefits, including:

1. Reduces redundancy.
2. Accelerates development.

3. Ensures that the design and behavior are consistent.

Part 2. Lists, Forms, & Events:

Lists, forms, and events are the building blocks of interactive web applications in JavaScript.

In JavaScript, Lists are known as Arrays, and they store ordered collection of data.

Events are actions that happen in a browser during user navigation, which includes mouse clicks, submitting forms, pressing on links, etc. (ex: `element.addEventListener('event_type', callback_function);`).

Rendering lists with .map() means taking an array of raw data and transforming it into a collection of visual UI elements.

This saves the developers the trouble of having to rewrite the code manually for each item, and instead do it once and call it multiple times.

Example:

The process of doing so is quite simple. Let's say we have this list

```
<ul>
  <li>Creola Katherine Johnson: mathematician</li>
  <li>Mario José Molina-Pasquel Henríquez: chemist</li>
  <li>Mohammad Abdus Salam: physicist</li>
  <li>Percy Lavon Julian: chemist</li>
  <li>Subrahmanyan Chandrasekhar: astrophysicist</li>
</ul>
```

First thing we'll do is put them in an array:

```
const people = [
  'Creola Katherine Johnson: mathematician',
  'Mario José Molina-Pasquel Henríquez: chemist',
  'Mohammad Abdus Salam: physicist',
  'Percy Lavon Julian: chemist',
  'Subrahmanyan Chandrasekhar: astrophysicist'
];
```

Then, we're going to **map** the people into a new array of JSX nodes, listItems:

```
const listItems = people.map(person => <li>{person}</li>);
```

After mapping these specific members, we will return **listItems** from the component wrapped in between `<ul></ul>`:

```
return <ul>{listItems}</ul>;
```

This is the outcome of the process:

App.js

```
1  const people = [  
2    'Creola Katherine Johnson: mathematician',  
3    'Mario José Molina-Pasquel Henríquez: chemist',  
4    'Mohammad Abdus Salam: physicist',  
5    'Percy Lavon Julian: chemist',  
6    'Subrahmanyam Chandrasekhar: astrophysicist'  
7  ];  
8  
9  export default function List() {  
10    const listItems = people.map(person =>  
11      <li>{person}</li>  
12    );  
13    return <ul>{listItems}</ul>;  
14  }
```

- Creola Katherine Johnson: mathematician
- Mario José Molina-Pasquel Henríquez: chemist
- Mohammad Abdus Salam: physicist
- Percy Lavon Julian: chemist
- Subrahmanyam Chandrasekhar: astrophysicist

(<https://react.dev/learn/rendering-lists>)

Keys and Controlled Components are core concepts in React that manage how elements maintain their identity, preserve state, and sync data between the UI and logic of JavaScript.

Form submission handling is the process of capturing a user's input and transmitting it to a server or API. This process includes intercepting, validating, and processing data entered by a user into an HTML form.

Input validation and multiple input handling:

1. Input validation: checks if data is safe, in the correct format, and meets specific criteria, such as password lengths and email patterns.
2. Multiple input handling: the structural mechanism that tracks, updates, and processes data from several different fields simultaneously on the UI.

Basically, input validation verifies that the data provided by the user is safe and meets the expectations or specifications. On the other hand, multiple input handling manages how applications capture and store data for multiple fields at once.

Part 3. Implementation

After reviewing the key points and my JavaScript vs TypeScript project, I will document below where the points are implemented:

First Draft:

Concept	Status
Functional Components	Applied
Props (properties)	Applied
useState hook	Applied
Event Handling	Applied
Conditional Rendering	Applied
Component Reusability	Applied
Rendering lists with .map()	Applied
Keys in React	Applied
Form Submission Handling	Missing
Single & Multiple Input Validation	Missing

TO DO:

1. Convert px to rem (font size) [DONE]
 2. Use padding to remove the space around the centered dashboard [DONE]
 3. Components: individual and shared ones => header, footer, and navbars [DONE]
 4. Divide these into different components [DONE]
 5. Deep understanding of the report + code + files [DONE]
-

Phase 2

Check list for phase 2:

useEffect & API Integration

- useEffect hook (lifecycle, side effects) [DONE]
- Fetching data from APIs, Loading states & Error handling [DONE]
- Critical: Connecting React to their WebAPI backend [DONE]
- CORS basics and how to handle it [DONE]

useEffect hook: used in React to handle side effects, which is anything a component needs to do that requires:

1. An external API.
2. Fetching data from a server.
3. Starting a timer.
4. Setting up a subscription.

In other words, it tells React to perform a specific action after rendering.

`useEffect(() => {})` → after every render.

`useEffect(() => {}, [])` → only once after the first render.

`useEffect(() => {}, [value])` → whenever `value` changes.

`return () => {}` inside `useEffect` → cleanup before unmounting or before the effect runs again.

CORS: Cross-origin resource sharing

Security feature in web browsers that restricts web pages from requesting data (ex: APIs) from a domain that's different from the one that served the original page.

How can we handle a CORS error?

This issue can be resolved by configuring the backend server and making it return the correct HTTP headers. These headers authorize the frontend domain to access its resources.

React Router - Part 1.1

React Router is a system that allows a react application to display different pages or views without reloading the entire website [Single Page Application].

Reloading the entire website happens by default in normal applications. The process behind it is: user clicks a link → request is sent from browser to the server → returns a whole new html page → full page refresh is caused.

However, the process in React Router is different; the parent content remains fixed in the main page, and only the changeable content (child) is updated [No page refresh].

Task:

- Installing React Router [DONE] Routes and navigation [DONE]
 - Link components [DONE]
 - Route parameters [DONE]
 - Nested routes [DONE]
 - Focus: Building multi-page applications [DONE]
 - Practice : Add routing structure to previous project [DONE]
-

React Router - Part 2

Task:

- Dynamic routes with parameters [DONE]
- Query parameters [DONE]
- Programmatic navigation (useNavigate) [DONE]
- Route guards/protected routes basics [DONE]
- 404 Not Found pages [DONE]
- Navigation state [DONE]
- Practice : Implement complete navigation system [DONE]

Req 1 - Dynamic route with parameters: flexible URL pattern that captures variable data (ID, slug) rather than relying on static hardcoded paths. Instead of hardcoding each page, [TopicView.tsx] and [LevelView.tsx] can both read the parameters in [App.tsx], and it renders any topic or level.

Req 2 - Query parameters [ArrivedViaSearchBanner.tsx, Ln13-14]: defined set of parameters attached to the end of a URL used to provide additional information to a web server when making requests.

Req 3 - Programmatic navigation [useNavigate]: the concept of useNavigate is making the website move on to the next level based upon completion ONLY.

Implementation:

1. [LevelView.tsx]: if the user answers a question correctly, only then can she/he unlock the next level.
2. [QuestComplete.tsx]: once the quest is completed, users can press “Play Again!”, and it’ll take them back to level 1.
3. [SearchBar.tsx]: calls navigate(...) to send them to their destination and attaches a tag in the URL that tells you the page title + a hidden note with the actual word the user typed. Finally, once the user is done, the search bar gets cleaned up. This follows a dynamic approach since the search is different every single time.

Req 4 - Route guards / protected routes: before moving on to the next page, it must check the conditions are met beforehand. I have implemented this in [RequiredLevelUnlocked.tsx] so the user cannot move on to the next level without clearing the current one.

Req 5 - 404 Not Found error [NotFound.tsx]: if the user types a URL that doesn’t match any of the routes that I have assigned, the error page will appear



Req 6 - Navigation state: a secret note that is sent along the navigation that doesn’t show up in the URL.

Param → part of the path that identifies what page we’re in.

Query → visible log on the URL - extra details, shareable, bookmarkable.

State → invisible data alongside the navigation - extra detail - refresh == gone.

Context API & Global State

Task:

- When to use global state vs local state [DONE]
- Creating Context [DONE]
- useContext hook [DONE]
- Provider pattern [DONE]
- Multiple contexts [DONE]
- Use case: User authentication state, theme, app settings [DONE]
- Practice : Implement global user authentication state [DONE]

When to use global state vs local state

Name	Local State	Global State
Usage	Used for data that affects only one component	Used when multiple distant components need to share and synchronize the same data user authentication
Examples	Form inputs UI toggles Modals	Dark / Light theme settings Global caches Shopping cart data

Req 1 - Creating Context

Generates a container that can hold a value and be read anywhere in the component tree below it without the value being passed down as a prop through every level. It uses the prop drilling approach where the context skips the middlemen entirely, meaning that if the **AccountPage** needs **user**, it would have to pass multiple layers down to the **AccountPage**.

Req 2 - useContext [**AuthContext.tsx**, **SettingsContext.tsx**, **ThemeContext.tsx**]

Allows a functional component to read and subscribe to context from anywhere in the component tree without manually passing props down through intermediate components (solves prop drilling).

Req 3 - Provider Pattern

Also solves prop drilling issue as it shares data across a global or nested component tree without passing props manually through every level.

It also syncs to **<localStorage>** via **<useEffect>**, so login state, theme, and settings survive a page refresh.

Req 4 - Multiple Contexts [AppProviders.tsx]

That file nests all four providers → Auth, Settings, Theme, QuestProgress.

Q / Why do we split separate contexts instead of using one big context?

Avoids unnecessary re-rendering: if auth, theme, and settings were one context, then toggling the theme would re-render every component that only cares about auth data. Splitting them means a component calling useTheme() doesn't re-render when settings change.

Week 5 - 06/07/2026 Monday

Part 1 - Advanced State Patterns:

State patterns in React refer to the standard strategies, design architectures, and best practices developers use to structure, update, and share data within an application.

Task:

- useReducer hook for complex state [DONE]
- Combining Context with useReducer [DONE]
- State management best practices [DONE]
- Avoiding prop drilling [DONE]
- When to lift state up [DONE]

Practice: Refactor application to use better state management [DONE]

Part 2 - Axios & API Integration

Axios is a popular, promise-based JavaScript library used to send asynchronous HTTP requests. It works seamlessly in both web browsers and Node.js environments.

Task:

- Installing and configuring Axios [DONE]
- Axios vs Fetch API [DONE]
- Creating Axios instances [DONE]
- Interceptors for adding auth headers [DONE]
- Error handling with Axios [DONE]
- Request/response transformation [DONE]

Practice: Replace Fetch with Axios and add interceptors [DONE]

Part 1 - Advanced State Patterns:

Req 1 - useReducer for complex state

What it is: **instead of useState**, we describe state changes as small named actions & a single function (**reducer**) that computes the next state from the current state + action.

Built-in function used to manage complex state logic.

(**useState is for light, simple projects. useReducer is for big, complex ones**).

Code implementation [[questProgressReducer.ts Ln13-33](#)] then [[QuestProgressContext.tsx Ln9](#)]

Req 2 - Combining Context with useReducer

What it is: **useReducer alone only manages state inside one component**. Context makes state available to any component in the tree. We put them together when several unrelated components need the same reducer-managed state.

Reducer decides how the state changes. Context decides who can see it.

<cleared> (finished levels) is needed by **5 separate components**. **useReducer is inside one of them only and can't reach the other, so:**

- 1. Context comes in to share it.**
- 2. useReducer keeps the updated logic clean once shared.**

We use both Context and useReducer together whenever state is complex and needed in multiple places.

Where it's implemented: [[QuestProgressContext.tsx Ln7-24](#)]

Reducer owns the logic behind how state changes.

Provider owns how it's exposed.

Req 3 - State management best practices

- 1. Not to use useReducer everywhere.** [AuthContext.tsx](#) and [ThemeContext.tsx](#) kept **useState one simple value**. No need to add reducer ceremony (**Reducer ceremony is the structured, multi-step process required to use the useReducer hook**).
 - 2. Memoize the Context value** → memoizing is giving memory to the code to prevent wasting time recalculating the exact same thing twice.
 - 3. Keep reducers in their own file with no React code** → this makes it easy for them to read and test on their own.
-

Req 4 - Avoiding prop drilling

How I avoided prop drilling in my project:

- 1. Quest state was drilled through the router instead of being used directly.**

[QuestLayout.tsx] was pulling quest data from Context, then repassing it down through React Router's outlet, & the three child routes [[LevelView.tsx](#)], [[QuestComplete](#)], and [[RequiredLevelUnlocked.tsx](#)] were digging it back out ⇒ This process == Prop Drilling.

Fix: removed the relay and had each child call the Context hook directly.

Req 5 - When to lift state up

Lift state up in React should be lifted when two or more components need access to the same changing data to ensure a single source of truth. By moving the state to their closest common ancestor, we allow data to flow down as props while updates pass up via callback functions.

Part 2 - Axios & API Integration

Req 1 - Installing and configuring Axios:

To install it I wrote in my terminal “npm install axios” then it got downloaded in my package.json dependencies.

Req 2 - axios vs fetch API:

Fetch and Axios are commonly used web applications to make HTTP requests from the frontend to backend.

Fetch → modern browsers.

Axios → simple library with extra features.

Fetch is a JavaScript feature available in modern browsers that uses the fetch() method to make network requests.

1. Fetch() → **make network requests and return a promise**
2. then() → **handle successful response**
3. catch() → **handles any errors that occur during the request**

Axios is a 3rd party JavaScript library designed to make HTTP requests, which works with both browsers and Node.js. Axios uses promise API introduced in ES6, and is known for its simplicity and extra features, such as:

1. **Request/response interception**
2. **Error handling**
3. **Support for cancellation**

-
1. axios.get() → **make a GET request to the specified URL and return a promise**
 2. .then() → **handle successful response**
 3. .catch() → **handle any errors that occur during the request**

Fetch is used for simple, light projects without the need of additional libraries.

Axios is used for heavier, complicated projects that require advanced features and a higher level of convenience when handling requests.

Req 3 - Creating axios instances

Creating an Axios instance means instead of using the plain, global `axios` object everywhere, we make our own copy of it with `axios.create({...})`, pre-loaded with settings we don't want to repeat.

Req 4 - Interceptors for adding auth headers

An interceptor is code that automatically runs on every request or response that goes through an Axios instance.

User doesn't call it, axios calls it for us

1. **Right before the request goes out.**
2. **Right after the response comes back.**

Request interceptor for auth headers specifically means: automatically attaching our login token to outgoing requests, so we don't have to remember to add it manually every single time we call an API.

[`httpClient.ts` Ln 40-55] Code Breakdown:

1. `httpClient.interceptors.request.use(...)` — "before you send any request through `httpClient`, run this function first."
2. `config` is the request that's about to be sent (its URL, headers, method, etc.) — this function gets a chance to inspect or modify it before it actually goes out.
3. `localStorage.getItem(AUTH_STORAGE_KEY)` — checks if someone is logged in, by reading the same key `AuthContext` uses to store the user.
4. If someone's logged in, `config.headers.set("Authorization", "Bearer <username>")` attaches an `Authorization` header — this is the standard way APIs check "who is making this request?"
5. `return config` — hands the (now-modified) request back to Axios so it can actually be sent.

Req 5 - Error handling with Axios

`httpClient.interceptors.response.use(...)` catches every failure and turns it into one consistent `ApiError`. In my case, I handle `ERROR 429 - Too many requests (user waits 1s)` & Network failure

Req 6 - Request/response transformation

the same interceptor reads the `Retry-After` header on a `429` response and converts it into milliseconds my retry logic can actually use.

Practice: Replace Fetch with Axios and add interceptors that's the whole `api.ts` rewrite- done.

Q/ Why did I use React Router DOM and not React Route?

I used **react-router-dom** instead of **react-router** because **react-router-dom** is specifically designed for web browsers.

It provides the exact components required to interact with the browser's URL and history API.

Week 5 - 07/07/2026 Tuesday

Authentication & Component Libraries

Authentication Implementation

- Login/logout functionality [DONE]
- Storing JWT tokens (localStorage vs sessionStorage) [DONE]
- Protected routes implementation [DONE]
- Auth context setup [DONE]
- Redirect after login [DONE]
- Token expiration handling [DONE]

Practice : Complete authentication system with their backend [DONE]

:User Session Management

- Persistent login (remember me) [DONE]
- Auto-logout on token expiration [DONE]
- Refresh token implementation (if backend supports) [DONE]
- User profile management [DONE]
- Authorization (roles and permissions) [DONE]

Practice : Enhance auth system with session persistence [DONE]

Part 1 - Authentication implementation

Req 1 - Login/logout functionality [contexts/AuthContext.tsx]

login is an **async** function; it sends the username/password to the backend and *waits* for a reply before doing anything else.

setIsLoading(true) flips a flag for the UI to read and disable the button then shows "Logging in..." while it waits.

If **loginRequest** throws (bad password, network error, whatever), the **catch** block turns that into a readable message and re-throws it, so the form component can also react if it wants to.

finally always turns **isLoading** back off, success or failure, so the button never gets stuck disabled.

-

logout is not **async** — notice there's no **await** on **logoutRequest**. It fires that request and immediately moves on to **endSession()**, which clears everything locally

`.catch(() => {})` on the request means "if this network call fails, silently ignore it — don't let it block anything."

Why was it implemented this way?

Login has to be a slow, careful process because we don't know if the user is authenticated until the server says so, so the UI has to wait and show an error.

Logout is the opposite as it should be instant.

Req 2 - Storing JWT tokens (Json web tokens- verify the user's identity and permissions after logging in) [`features/auth/session.ts`]

`localStorage` and `sessionStorage` have the exact same API (`.setItem`, `.getItem`, `.removeItem`) the only difference is when the browser clears them → `localStorage` survives closing the tab/browser, `sessionStorage` doesn't.

Because they share an API, `target` can just be a variable pointing at whichever one applies, based on the `persist` boolean — no need to write the save logic twice.

`other.removeItem(...)` deletes any leftover copy in the *other* storage, so a token never silently exists in both places at once (which would make "am I remembered?" ambiguous later).

Req 3 - Protected routes implementation [`auth/RequireAuth.tsx Ln13-28`]

This is a wrapper component. It doesn't render `AccountPage` at all unless `isAuthenticated` is true.

`<Navigate>` in React Router's way of saying redirect the browser to this URL.

It is considered a component and not a function call because React Router needs it to happen as part of the rendering, not side effect.

`<replace>` means it swaps the current history entry instead of adding a new one, so hitting the browser's Back button after logging in doesn't take you back to the "blocked" state.

No rendering occurs when the user is logged out, it leaves the user stuck on a blank page.

Redirecting to `/login` route means the URL bar reflects what's happening and that the same pattern is used somewhere else in the same application. (same concept as the 3 levels and having to complete one to move on to the next.

Req 4 - Auth context setup

`createContext(null)` makes a "box" that can hold a value and be read from anywhere below it in the component tree, without passing it down through every component's props by hand.

`AuthProvider` is the component that fills that box (it holds the real `session` state and computes a `value` object to put in the box, and any value that calls `useAuth()` pulls the value back out).

Error handling: The `if (!ctx)` throw guard exists so that if a user ever accidentally uses `useAuth()` outside of `<AuthProvider>`, they get a clear error immediately instead of a confusing `undefined.something` crash somewhere else.

Req 5 - Redirect after login

[auth/RequireAuth.tsx] → stashes where you were headed (hides)

[auth/LoginPage.tsx] → reads it back

React Router lets you attach any data to a navigation via **state**- it's not visible in the URL, it just rides along with that specific navigation.

when **RequireAuth** bounces you to **/login**, it also hands over "here's the page you were actually trying to reach" (**location**, which includes the pathname).

Why do we use this instead of a URL?

A query string is visible and editable in the address bar + needs manual encoding if the path has special characters.

Router state is invisible, disappears naturally after use, and can't be tampered with by just editing the URL.

Req 6 - Token expiration handling [auth/jwt.ts]

Meaning: creating a system that detects when a user's security token (like a JWT) becomes invalid and seamlessly updates it or logs the user out without crashing the app or confusing the user.

JWT can look like: **header.payload.signature** 3 base64 encoded chunks separated by dots.

decodeJwt reads the middle chunk → middle chunk == JSON object from the backend.

Implementation notes:

JWT stores **exp** in seconds, but JavaScript's **Date.now()** works in milliseconds, which is why **getExpiryTime** multiplies by 1000.

isTokenExpired is like saying "is right now past that time?"

Data Tables & Pagination

- Displaying large datasets efficiently [DONE]
 - MUI Data Grid or Ant Design Table or React Table [DONE]
 - Server-side pagination implementation [DONE]
 - Sorting and filtering [DONE]
 - Search functionality [DONE]
 - Column customization [DONE]
- Practice : Build data table connected to backend with pagination [DONE]

Req 1 - Displaying large datasets efficiently [features/library/DataTablePage]

Pagination meaning: a web development technique used to split a massive dataset into smaller, discrete chunks (pages) instead of rendering all data simultaneously.

The table doesn't render the whole dataset at once. Only the current page's rows (**rows**) are held in component state and passed to the grid. [Ln 58-59].

Component state: built-in object used to store data that changes over time and directly influences what appears on the screen. When a state variable is modified, React triggers an automatic re-render of that specific component and its children to keep the user interface in sync with the new data.

Q/ Why is rowCount tracked separately from rows.length?

-rowCount represents the total number of matching records across all pages.

-Rows only hold 1 page's worth of data.

→ This is what lets the grid show "1–10 of 20" style pagination controls without ever holding all 20+ rows in memory at once — the same pattern is what makes this scale to thousands of rows without a performance hit, since the browser only ever renders one page.

Req 2 - MUI Data Grid [features/library/DataTablePage Ln4-7]

```
import {
  DataGrid,
  type GridColDef,
  type GridPaginationModel,
  type GridSortModel
} from "@mui/x-data-grid";
```

MUI: Material-UI is an open-source React component library that provides pre-designed, production-ready user interface (UI) elements. It drastically speeds up web development by allowing developers to import ready-made elements instead of coding CSS from scratch. (DataGrid) is the table component itself.

Why did we choose this method instead of a plain html `<table>`?

Using `DataGrid` from MUI's `x-data-grid` package comes with pagination, storing, and column resizing built in. It simplifies the dev's job; instead of implementing grid behavior from scratch, we only have to supply data and configuration.

(`GridColDef`) types the column definitions.

(`GridPaginationModel` & `GridSortModel`) type the pagination and sorting state that get passed back and forth with the grid.

Req 3 - Server-side pagination implementation [`features/library/DataTablePage.tsx`]

Ln201: `paginationMode="server"` tells the `DataGrid` "don't paginate the data yourself — I'll hand you exactly the rows for the current page, you just render them and show the page controls."

- `paginationMode="server"`: paginates data & provides rows for the current page.
- `DataGrid`: renders them.

Ln204: Whenever the user clicks a page number or changes page size, `onPaginationModelChange` fires and updates `paginationModel`, which triggers a `useEffect` that refetches from `adventurersAPI.ts`

Q/Why did we choose server-side instead of client-side pagination?

with client-side pagination, the grid would need the *entire* dataset in the browser and would just hide/show rows locally. Server-side pagination instead only ever transmits and holds one page of data at a time, which is what makes this pattern relevant to "large datasets" — it's the same shape a real API call (`GET /adventurers?page=2&pageSize=10`) would take, just backed by an in-memory array with an artificial delay (`SIMULATED_LATENCY_MS`) instead of a network request.

Req 4 - Sorting and filtering [`library/data/DataTablePage.tsx`]

Sorting is wired the same way as pagination:

- grid owns the UI (clicking a column header).
- app owns the actual sort.

Code Explanation:

1. `sortModel` — which column is sorted and in which direction. It's an array because MUI's `GridSortModel` type supports multi-column sort, though this table only ever uses the first entry.
2. `status` — the current filter selection from the "Status" dropdown ("all", "hero", "villain", or "ally").
3. `searchInput` — the raw text currently typed into the search box, updated on every keystroke.
4. `search` — the "settled" search term that's actually sent to the API. It lags behind `searchInput` on purpose.

```
useEffect(() => {
  const timeout = setTimeout(() => setSearch(searchInput), 300);
  return () => clearTimeout(timeout);
}, [searchInput]);
```

Debounce: `searchInput` updates on every keystroke (so the text field feels instant), but `search` (the value actually sent to the fake API) updates 300ms after the user stops typing. Without this, every keystroke would trigger a new fetch, which is wasteful and would cause visible flicker/race conditions as older requests resolve after newer ones.

Req 5 - Column Customization

Each column definition demonstrates a different customization option `GridColDef` supports:

- `headerName` decouples the column's internal field key (`adventurerClass`) from what's shown to the user ("Class").

Decouple: reducing the dependencies between different parts of a software system. Instead of having tightly connected components where changing one breaks another, this allows individual modules to be tested, updated, and maintained independently.

- `width` vs `flex` — most columns get a fixed pixel `width`, but `name` uses `flex: 1` so it grows to fill any remaining horizontal space, since names vary a lot in length and benefit from extra room.
- `type: "number"` tells the grid to right-align the column and sort/filter it numerically instead of as text.
- `sortable: false` on `status` disables the sort-by-click behavior for that column, since sorting by an arbitrary hero/villain/ally label wouldn't be meaningful.
- `renderCell` replaces the default plain-text cell with custom JSX — here, an MUI `Chip` whose label and color both come from lookup tables (`statusLabels`, `statusColors`) keyed by status, so each of the three statuses gets distinct, readable styling instead of plain text.

Component Libraries (Advanced)

- Data display components (Table, List) [DONE]
- Feedback components (Alert, Snackbar, Dialog) [DONE]
- Navigation components (AppBar, Drawer, Tabs) [DONE]
- Form components (Select, Checkbox, Radio) [DONE]
- Customizing component styles [DONE]

Practice : Build a complete dashboard layout with library components [DONE]

Req 1 - Data display components (table, list) [**ComponentsPage.tsx**]

Lists: **Ln109-130**

Table: **Ln154-173**

It shows the pop up menu sidebar at library/components (drawer).

Req 2 - Feedback components (Alert, snackbar, dialog) [**ComponentsPage.tsx**]

Alert [**Ln134**]:

Snackbar [**Ln230**]: temporary UI component that displays brief messages after saving or resetting settings. It auto-dismisses after 2.5seconds.

Snackbar: handles the timing and position.

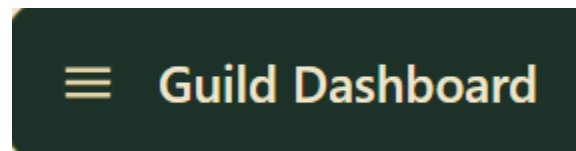
Alert within snackbar: supplies the colored message content.

Dialog [**Ln241**]: a floating UI window that appears temporarily on top of the main web page content. In my implementation, it's a blocking modal used specifically because resetting is destructive → it forces an explicit Cancel/Reset choice rather than happening instantly.

Req 3 - Navigation components (AppBar, Drawer, Tabs) [**library/ComponentPages**]

AppBar [**Ln86**]: main navigation or action bar. Typically positioned at the top of the screen displaying branding, current page title, navigation icons, and quick-action buttons (like search, settings, or user profiles).

Drawer [**Ln107**]: slide-in-side panel (opened by the menu icon) with links to other pages.



Tabs [**Ln94**]: which panel to render.

Req 4 - Form Components (Select, Checkbox, Radio) [**library/ComponentPages**]

Select [**Ln194**]: choose which difficulty you want- collapses after selection.

Checkbox [**Ln213**]: independent on/off toggle.

Radio [**Ln204**]: few, mutually-exclusive options shown all at once side-by-side (**row**), since there are only two (light/dark) and a dropdown would be overkill.

Q/Why did we use 3 components?

Many options → Select then collapse

Few visible options → Radio

One independent toggle → Checkbox

Req 5 - Customizing component styles

2 layers working together:

1. Global theme (src/shared/mui/muiTheme.ts Ln56)

Applies to every AppBar in the app automatically (That's why the one in ComponentPage.tsx never sets its own background color).

2. Per-instance (features/library/ComponentsPage.tsx Ln86):

Rounds the specific AppBar corners without affecting any other AppBar elsewhere.

Q/Why do we use both layers?

Global theme includes anything that should look consistent everywhere (defined once)

Per-instance includes small, situational tweaks meant for one specific spot on the page.

That way both global theme and per instance stay contained to that single element instead of accidentally affecting every other place where that same component is used elsewhere in the app.

Practice: Build a complete dashboard layout with library components [ComponentsPage] ComponentsPage.tsx is the answer. It's a one working page that combines navigation, data display, feedback, and forms.

Week 6 - 13/07/2026 Monday

NextJS Notes

NextJS is a React framework.

React only builds the UI, while Next.js is a full-stack framework built on top of React that adds structure, backend capabilities, and optimizations for a complete, production-ready web application.

Some of NextJS's additional features include:

- 1. Routing (pages)**
- 2. Backend APIs**
- 3. Better performance**
- 4. SEO**
- 5. Authentication support**
- 6. Image optimization**
- 7. Server-side rendering**
- 8. Deployment**

Q/ When to use React and when to use NextJS?

React: Ideal for highly interactive, private software like single-page applications (SPAs) behind a login screen, internal business dashboards, or tools where search engine indexing (SEO) does not matter.

Next.js: Ideal for public-facing websites where search rankings and loading speeds directly impact user metrics, such as e-commerce storefronts, blogs, marketing landing sites, and content-rich platforms.

Q/ What is SEO?

Search engine optimization is the practice of configuring your website so that search engines like Google can easily understand and rank your pages highly in search results.

In NextJS, we don't need react libraries or React Router. It all depends on the file structure. For example, every folder inside app becomes a URL.

Difference between React and NextJS when fetching data:

Example 1 - Links:

React Router simply calls a link by → `<link to="/about">`

NextJS → `import Link from "next/link";`

```
<Link href="/about">
```

```
  About
```

```
</Link>
```

Example 2 - Images

React Router → ``

NextJS → `import Image from "next/image";`

```
<Image
```

```
  Src = "/cat.jpg"
```

```
  Width={300}
```

```
  height={300}
```

```
  alt="cat" />
```

Why does NextJS follow this method?

This method automatically:

- ★ compresses images
- ★ lazy loads
- ★ speeds loading
- ★ resizes images

Lazy loads is an optimization technique where we purposely delay loading specific pieces of code or components until they are needed by the user.

Client Components vs Server Components

In React, everything runs in the browser.

In NextJS, some things run on the server first == faster

We use Client Component if we need:

- `useState`

- **useEffect**
- **onClick**
- **Browser APIs**

However, we need to ensure we apply this otherwise **useState** cannot be used in a server component:

“use client”;

```
Export default function Counter() {
}
```

Types of Rendering:

1. CSR (Client-side rendering)

The browser downloads JavaScript first then builds the page.

Advantage: good for dashboards & highly interactive applications.

Disadvantage: Slower first load and weaker SEO.

2. SSR (Server-side rendering)

The server builds the page for every single request

Advantage: Great SEO and always fresh data.

Disadvantage: Slightly more server work.

Feature	 React	 Next.js
 What it is	A JavaScript library for building user interfaces.	A React framework with built-in features for full-stack web apps.
 Routing	Requires React Router or another routing library.	Built-in file-based routing (app/about/page.tsx → /about).
 Setup	You configure routing, bundling, SEO, etc., yourself.	Most features are preconfigured and ready to use.
 Rendering	Primarily Client-Side Rendering (CSR).	Supports CSR, SSR, SSG, and ISR.
 Server Components	✗ Not available.	✓ Available by default in the App Router.
 Client Components	Everything is a client component.	Components are server components by default; add “use client” when needed.
 Data Fetching	Usually <code>useEffect()</code> + <code>fetch()</code> .	Can fetch data directly in async server components.
 Backend/API	Requires a separate backend (Express, Node.js, etc.).	Can create backend endpoints using <code>app/api/.../route.ts</code> .
 SEO	Requires extra libraries or manual configuration.	Built-in metadata and server rendering improve SEO.
 Image Optimization	Uses standard <code></code> tags.	Uses optimized <code><Image></code> component with lazy loading and resizing.
 Navigation	Uses <code>react-router-dom</code> with <code><Link to=""></code> .	Uses <code>next/link</code> with <code><Link href=""></code> .
 Layouts	Must manually include <code>Navbar/Footer</code> on each page or wrap components.	Built-in <code>layout.tsx</code> automatically wraps pages.
 Code Splitting	Manual or configured through bundlers.	Automatic code splitting by route.
 Performance	Depends on your optimization.	Many optimizations are built in.
 Deployment	Can be deployed anywhere as a frontend app.	Optimized for platforms like Vercel, but can also run on other Node.js hosts.
 TypeScript Support	Optional.	Built-in support with easy setup.
 Project Structure	Flexible; you choose the folder organization.	Convention-based (app/, components/, public/, etc.).
 Learning Curve	Easier to start.	Slightly steeper because of server/client concepts.
 Best For	Single Page Applications (SPAs), dashboards, internal tools.	Production websites, e-commerce, blogs, SaaS apps, and SEO-focused applications.

How to convert from React to NextJS?

First thing we need to do is replace our old React build scripts (like vite or react-scripts) with Next.js → delete the existing tool (npm uninstall vite @vitejs/plugin-react or npm uninstall react-scripts) and replace it with the latest version of NextJS.

Lastly, we're gonna update the scripts in the package.json file:

```
"scripts": {  
  "Dev": "next dev",  
  "Build": "next build",  
  "Start": "next start",  
  "Lint": "next lint"  
}
```

Week 6 - 13/07/2026 Monday

Document the difference between React and NextJS

Point 1 - What is the difference between React and NextJS?

React only builds the UI

Next.js is a full-stack framework built on top of React that adds structure, backend capabilities, and optimizations for a complete, production-ready web application.

Some of NextJS's additional features include:

9. Routing (pages).
10. Backend APIs.
11. Better performance.
12. SEO.
13. Authentication support.
14. Image optimization.
15. Server-side rendering.
16. Deployment.

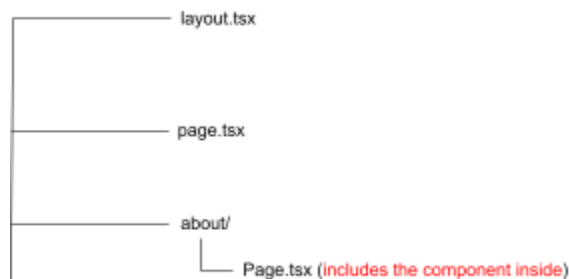
Point 2 - POSTPONED UNTIL FURTHER NOTICE

Point 3 - Understand the app folder

The app directory serves as the foundation for NextJS router.

App allows us to create a route. The process is:

1. Create a folder named "route".
2. Render a component inside of a page.tsx file inside it. (more scalable)



Page router has a folder called **pages**. When we put a file or folder inside of it, its name will become a route.

#components in the *app* folder are server components, meaning **they run on a server and their only task is to be able to send html to the client very fast.**

Everything in the *app* folder becomes a route in the website

Point 4 - page.tsx and layout.tsx

PAGE: Inside every route there must be a page. Ex: *app – products – page.tsx*:

app/products/page.tsx becomes the webpage.

Note: the folder (*products*) alone does nothing. It needs *page.tsx* to actually be able to call and reach the directory and view its content (*localhost:3000/products*)

LAYOUT: In React we have to manually repeat the code on every page, but in NextJS *layout.tsx* wraps pages underneath it (It solves the repetition of manually writing each code).

Layout.tsx is at the root of the *app* folder, meaning that every page uses it. The parent is fixed, but children are the part that changes.

Summary of *layout.tsx*:

- wraps shared UI (header, footer, navbars) around pages.
- The layout itself remains fixed, while the children (current page) change depending on the route.
- The *layout.tsx* at the root of the *app* folder is called *Root Layout*, but we can also create additional *layout.tsx* files inside the subfolders to share UI only within that section of the application.

Point 5 - File-based routing in NextJS

File-based routing means that NextJS automatically creates routes (URLs) based on my folder and file structure.

Unlike React, I don't have to manually define these routes.

Q/ What is the difference between routing in React and NextJS?

React → define every route using React Router. To add a new page:

1. Create the component
2. Import the component
3. Add a new `<route>` [check code example]

```
import { Routes, Route } from "react-router-dom";

<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/about" element={<About />} />
  <Route path="/products" element={<Products />} />
</Routes>
```

NextJS → Simply create folders and page.tsx files

```
app/  
|  
├─ page.tsx  
├─ about/  
|   └─ page.tsx  
├─ products/  
|   └─ page.tsx
```

```
/   
  
/about  
  
/products
```

NextJS will automatically create the 2nd image. No routing code is needed.

Q/ How does routing work in NextJS?

1. NextJS will scan the *app* folder.
2. Each folder becomes part of the URL.
3. Each page.tsx becomes the page displayed at that URL.

In other words:

Folder name → URL Segment.

Page.tsx → Webpage shown at that URL.

#Nested routes also use file-based routing. The difference between this example and the one under nested routes is:

Example 1 → pages are independent (directly inside app).

Example 2 → parent with children concept. Since *phones* and *laptops* are inside *products*, their URLs begin with */products*.

Point 6 - Nested Routes

```
app  
|  
├─ products  
|   |  
|   └─ phones  
|       |  
|       └─ iphone  
|           |  
|           └─ samsung  
|               |  
|               └─ laptops  
|                   |  
|                   └─ dell  
|                       |  
|                       └─ hp
```

Resulting URLs	
Folder	URL
app/page.tsx	/
app/products/page.tsx	/products
app/products/phones/page.tsx	/products/phones
app/products/phones/iphone/page.tsx	/products/phones/iphone
app/products/phones/samsung/page.tsx	/products/phones/samsung
app/products/laptops/page.tsx	/products/laptops
app/products/laptops/dell/page.tsx	/products/laptops/dell
app/products/laptops/hp/page.tsx	/products/laptops/hp
app/about/page.tsx	/about
app/contact/page.tsx	/contact

Theoretical example:

if I want to go to *hp*, the URL will include *laptops* and *dell*: *products/laptops/hp*.

Point 7 - Dynamic routes

Suppose my database contains 5000 phones. I will definitely not hardcode every single one. Instead, I will create [id] and anything can go there (products/[id]/page.tsx).

This way it can match anything: *products/1*, *products/50*, *products/100*, *products/iphone*, and all are used in one page.

The value is stored inside **params**, so if the user wants to visit *products/50* then the **params.id = 50**. OR if the user wants to visit *products/laptop* then the **params.id = laptop**.

Point 8 - Navigation using Link

In React, we often use `<a href="/about">`, but this reloads everything unnecessarily.

NextJS solution:

```
<Link href="/about">
```

About

```
</Link>
```

This way only the page will change. Navbars, footers, and JS state stays + it's faster.

Hidden feature → *prefetching*, meaning that if the About page is visible on the screen, NextJs quietly downloads it in the background, so when the user clicks it it appears almost instantly.

Prefetching: performance optimization technique where the framework downloads the assets, data, and code for a linked route in the background *before* the user actually clicks on it.

Point 9 - Navigation using useRouter()

Link only works when the user clicks on it.

Sometimes we need navigation after logic.

Theoretical example:

1. User clicks login
2. System checks password
3. If correct, go to dashboard

Same example implemented in code:

```
Const router = useRouter();  
router.push("/dashboard")
```

Some useful methods used:

1. **push()** → go somewhere (**router.push("/about")**)
2. **replace()** → useful after login
3. **back()** → browser back
4. **forward()** → browser forward
5. **refresh()** → reload current data, not the entire browser

Point 10 - Not Found page
We create `not-found.tsx`